

Práctica: Detección de anomalías en entornos inteligentes (CASAS scripted-ADL)

Entornos Inteligentes - Grado en Inteligencia Artificial, UA

1. Objetivo

Aprender a detectar anomalías (errores en la ejecución de actividades cotidianas) a partir de las activaciones de sensores de una casa inteligente, formulando la tarea, representando la señal, entrenando modelos de deep learning y validando los resultados.

Objetivos específicos:

1. Codificar eventos de sensores discretos en una representación tensorial fija (canales × tiempo) apta para redes neuronales.
2. Formular la tarea de detección de anomalías y reportar las métricas fijas de la sección 6 (AUC-ROC, F1, Precision, Recall, accuracy).
3. Comparar e implementar al menos una arquitectura de deep learning (MLP, CNN 1-D, GRU, Transformer, ResNet 1-D, etc.).
4. **Interpretar** sus resultados (no solo reportar cifras).

2. Contexto: ¿qué es detectar anomalías aquí?

En la vida real (por ejemplo, monitorizar la autonomía de una persona mayor) no se sabe de antemano qué es un “error”: la detección de anomalías pura consiste en modelar lo normal y alertar cuando la conducta se desvía (one-class, autoencoders de reconstrucción, distancia a lo normal). En esta práctica las anomalías están **definidas y etiquetadas**: son 5 errores.

Formulación operativa: dada una secuencia de activaciones de sensores de una ejecución, el modelo debe responder **normal** o **anomalía** (clasificación binaria supervisada de anomalías conocidas), idealmente indicando cuál es el error.

La anomalía se manifiesta en la señal de sensores. Ejemplo: si el canal de nivel de agua del fregadero no baja al final de la ejecución, alguien ha dejado el grifo abierto. El detector debe aprender este tipo de patrones temporales.

3. Dataset (Zenodo 15712834)

220 ejecuciones de 5 actividades de la vida diaria (ADL) guiadas en el apartamento Chinook ([CASAS, Washington State University](#)): dos sets, *adl_noerror* (120 ejecuciones normales, 24 por tarea) e *adl_error* (100 ejecuciones con error scriptado, 20 por tarea). Los participantes de ambos sets son distintos. Cada ejecución es un CSV *pXX.tY.csv* (participante × tarea) con columnas *date*, *time*, *sensor*, *message*; cada fila es un evento (12–241 eventos por ejecución; duración 0,4–14,6 min).

Tarea	Descripción	Error (la anomalía)
t1 - Llamar por teléfono	Ir al teléfono, buscar el número y llamar	Marca un número equivocado y debe volver a marcar
t2 - Lavarse las manos	Lavarse en el fregadero con jabón	No cierra el grifo al terminar
t3 - Cocinar	Cocinar avena siguiendo la receta	No apaga la hornilla tras cocinar

t4 - Comer	Lleva la comida y la medicación al comedor	No lleva el bote de medicación
t5 - Limpiar	Lavar los platos en el fregadero	No usa agua al lavar los platos

Sensor(es)	Tipo	Significado
M01–M026	PIR (discreto)	Movimiento: ON / OFF
I01–I08	Uso de objeto (discreto)	Objeto: PRESENT / ABSENT (avena, pasas, azúcar, bol, cucharín, bote de medicación, olla, libreta de teléfonos)
D01	Puerta (discreto)	Armario de cocina: OPEN / CLOSE
asterisk	Teléfono (discreto)	Uso del teléfono: START / END
AD1-A, AD1-B	Nivel analógico	Agua del fregadero (valor = nivel)
AD1-C	Nivel analógico	Hornilla (valor = nivel)
E01	Instrucciones (discreto)	START_INSTRUCT / STOP_INSTRUCT

Cita obligatoria: Cook, D. & Schmitter-Edgecombe, M. (2009). Assessing the quality of activities in a smart environment. *Methods of Information in Medicine*, 48(5), 480–485. doi:10.3414/ME0592.

4. El tensor fijo (entrada del detector): ejemplo

La clave de esta práctica está en esta codificación (la que tendrás que implementar en el Paso 2): cada ejecución se codifica en un tensor fijo de **30 canales × 180 bins de 5 s** (15 min), anclado al primer evento, con *carry-forward*: cada canal mantiene su último valor conocido hasta que llega un nuevo evento de ese sensor. Ejemplo con una ejecución real, **p22.t2** (tarea t2, con error: grifo abierto). Su CSV tiene 15 eventos en 31 s, pero el tensor tiene siempre la misma forma: 30 × 180. Solo 6 canales toman valores distintos de cero; los otros 24 son todos 0 en esta ejecución.

Canal	bin 0	bin 1	bin 2	bin 3	bin 4	bin 5	...	bin 179
M13 (PIR)	1	0	0	0	0	0	...	0
M14 (PIR)	1	0	0	0	0	0	...	0
M15 (PIR)	1	0	0	0	0	0	...	0
M16 (PIR)	1	0	0	0	0	0	...	0
M17 (PIR)	0	1	0	0	0	0	...	1
AD1-B (agua)	0	0.07	0.37	0.37	0.86	0.96	...	0.96

Dos detalles que hay que entender bien.

1. El bin 3 de AD1-B repite 0.37 porque no hay evento nuevo de ese sensor en ese bin: se mantiene el último valor conocido (*carry-forward*). El mismo mecanismo se ve en M17: se enciende (1) en el bin 1, se apaga (0) en el bin 2 y se mantiene en 0 hasta que vuelve a encenderse (1) en el bin 6. (2) La ejecución dura 31 s (≈ 6 bins), así que del bin 6 al 179 no hay más eventos: cada canal se mantiene en su último valor. AD1-B queda en 0.96 y M17 en 1. En una ejecución **normal** de t2 el nivel de agua baja al final (el grifo se cierra): **esa cola es donde vive la anomalía** y es lo que el detector debe aprender a leer. La etiqueta de p22.t2 es **y = 1** (pertenece al set *adl_error*); la de una ejecución normal, p. ej. p50.t2, es y = 0.

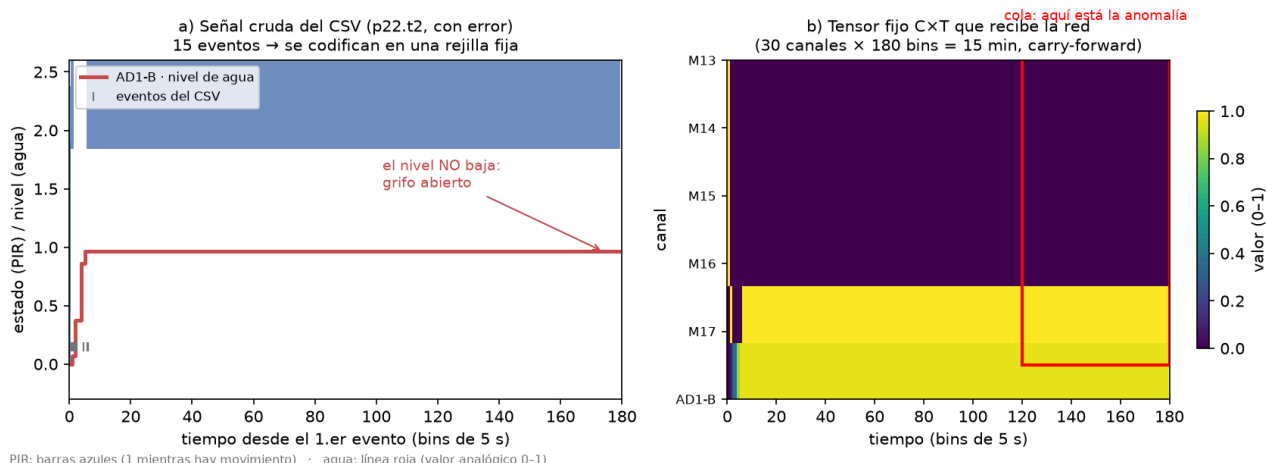
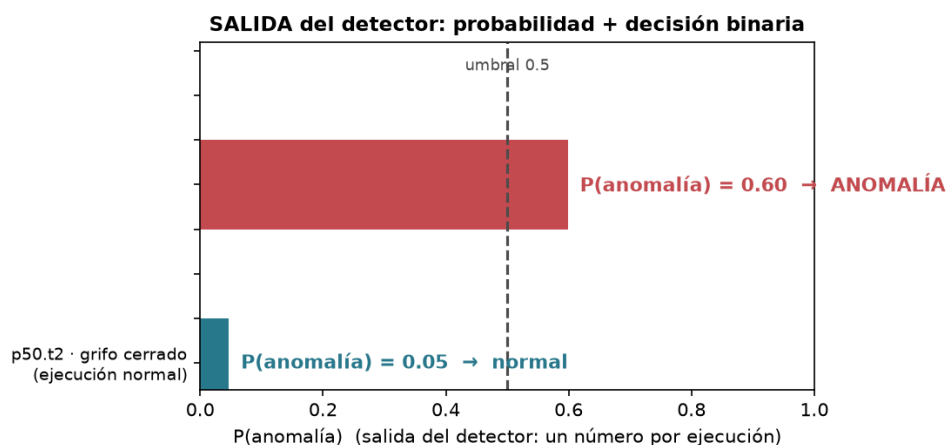


Figura 1. Entrada: a) señal cruda de p22.t2 (los 15 eventos del CSV; el nivel de agua no baja porque el grifo se queda abierto) y b) el tensor fijo 30 × 180 que recibe la red (*carry-forward*). La cola resaltada en rojo es donde se manifiesta la anomalía.



Ejemplo real: GRU entrenado con las otras 218 ejecuciones (p22 y p50 fuera del entrenamiento).

Figura 2. Salida: el detector devuelve un solo número por ejecución - la probabilidad de anomalía - y se decide con umbral 0.5. Valores reales de un GRU entrenado con las 218 ejecuciones restantes (p22 y p50 fuera del entrenamiento): la ejecución con error sale por encima del umbral y la normal, muy por debajo.

5. Consigna (paso a paso)

Paso 1 - Entender los datos. Explorar los CSVs de ambos sets: ¿qué sensores intervienen en cada tarea? ¿Cómo se ve en la señal el error de “dejar el grifo” (t2) o el de “no apagar la hornilla” (t3)? Dibuja al menos una pareja de ejecuciones (una normal y una con error) a nivel de sensor e inclúyela en el informe.

Paso 2 - Representar la señal. Codifica cada ejecución en un tensor fijo de C canales × T bins de tiempo, p. ej. una rejilla de 180 bins de 5s (15 min) anclada al primer evento, con *carry-forward* del

último estado conocido por sensor (los niveles analógicos se mantienen hasta el siguiente evento).
Pregunta clave a responder en el informe: ¿por qué el final (“cola”) de la ventana es donde se manifiesta la anomalía? ¿Qué pasa si no haces *carry-forward*?

Paso 3 - Entrenar un detector. Elige al menos una arquitectura (MLP sobre el tensor aplanado, CNN 1-D, GRU, Transformer temporal o similar) y entrénala con pérdida binaria (BCE con logit). Define el split: cross-validation por participante (ningún participante puede aparecer a la vez en entrenamiento y validación). Documenta hiperparámetros, semilla y tiempo de entreno.

Paso 4 - Evaluar sin trampas. Reporta el set de métricas fijo de la sección 6 (matriz de confusión; AUC principal, y F1, precisión, recall y exactitud al umbral 0.5), calculadas sobre las predicciones de la validación cruzada. No se aceptan métricas alternativas como cifra principal: pueden discutirse en el informe, pero la tabla de resultados usa siempre este formato.

Paso 5 - Informar. Informe (PDF) con: descripción de la señal y de la anomalía a nivel de sensor (figuras), protocolo de validación, resultados (tabla de métricas de la sección 6: AUC media +- desviación en la validación cruzada, y F1/precisión/recall/exactitud al umbral 0.5), análisis de fallos (¿qué tipos de error detecta bien y cuáles no?), y limitaciones del enfoque.

5.1 Test conjunto de la clase (fuera de todo entrenamiento)

Además de la validación cruzada, hay un test conjunto para toda la clase: 40 ejecuciones (8 participantes × 5 tareas; 20 normales y 20 con error) que nadie entrena. Quedan fuera del pool de la validación cruzada, así que ningún equipo las ve etiquetadas en ningún momento.

Protocolo fijo:

1. Entrena tu red una última vez, una sola vez, con las 180 ejecuciones restantes (semilla 42, mismos hiperparámetros que en el Paso 3).
2. Calcula las 40 probabilidades de anomalía y guárdalas en `pred_test.json` con el formato `{"equipo": "...", "model": "...", "pred": {"pXX.tY.csv": prob, ...40 filas...}}` (una probabilidad perteneciente [0,1] por ejecución, con el nombre exacto del fichero CSV).
3. Entérgalo. El AUC de `pred_test.json` sobre las etiquetas reales es la cifra que decide el ranking definitivo de la clase y el puntuable extra (sección 7). Como el test es idéntico e idénticamente entrenado para todos, la cifra es comparable entre equipos y no se puede “afinar”.

6. Métricas de evaluación (fijas para todos los equipos)

Para que los resultados de todos los equipos sean comparables, **todas** las arquitecturas se evalúan con el mismo conjunto de métricas, calculadas sobre las predicciones de la validación cruzada (agregadas entre folds) con umbral 0.5 sobre la probabilidad de anomalía. Todos los términos se definen a partir de la matriz de confusión:

	Predicho: normal	Predicho: anomalía
Real: normal	Verdad negativo (TN)	Falso positivo (FP)
Real: anomalía	Falso negativo (FN)	Verdad positivo (TP)

Y las métricas que se reportan (fórmulas sobre la matriz anterior):

Métrica	Fórmula	Qué mide
AUC (ROC)	Área bajo la curva ROC: probabilidad de que una anomalía aleatoria obtenga mayor puntuación que una normal aleatoria	Calidad de la ordenación; independiente del umbral
F1	$2 \cdot TP / (2 \cdot TP + FP + FN)$	Equilibrio entre precisión y recall al umbral elegido
Precisión	$TP / (TP + FP)$	Fracción de las alarmas que son verdaderas
Recall (sensibilidad)	$TP / (TP + FN)$	Fracción de anomalías detectadas
Exactitud (accuracy)	$(TP + TN) / (TP + TN + FP + FN)$	Fracción global correcta; engañosa con clases desbalanceadas

AUC es la métrica principal (se reporta como media +- desviación de los folds de la validación cruzada); F1, precisión, recall y exactitud se reportan al umbral 0.5. El ranking de arquitecturas (sección 7) se decide por AUC. No se aceptan métricas alternativas como cifra principal: pueden discutirse en el informe, pero la tabla de resultados usa siempre este formato:

Arquitectura	AUC (media $\pm$ std)	F1 @0.5	Precisión @0.5	Recall @0.5	Exactitud @0.5
(arquitectura 1)	...	...	...	...	...
(arquitectura 2)	...	...	...	...	...

7. Puntuable extra: ranking de arquitecturas (+1 punto)

Implementa **al menos dos arquitecturas distintas** (mejor tres o más: p. ej. MLP, CNN 1-D, GRU, Transformer) **sobre el mismo protocolo** (misma semilla, mismo split, mismo optimizador y mismo número de épocas) y presenta un ranking por AUC en validación cruzada. El ranking no basta: en el informe debes **interpretar por qué** cada arquitectura gana o pierde (patrones locales vs. estado recursivo vs. atención a largo plazo) y discutir la varianza (el dataset es pequeño: 220 ejecuciones). Se valoran modelos con tamaño justificado frente al overfitting.

Quién se lleva el +1: tras recopilar los pred_test.json de toda la clase, se ordenarán las arquitecturas por AUC en el test conjunto (ranking definitivo). El +1 corresponde al mejor resultado de la clase: la arquitectura (y su equipo) que quede 1.^ª en ese ranking, es decir, la de mayor AUC sobre las 40 ejecuciones de todo el grupo. No basta con probar varias arquitecturas: el punto extra se lo lleva quien mejor haga contra el resto de la clase. En caso de empate a AUC, se rompe con el F1 a umbral 0,5 y, a continuación, con la exactitud a umbral 0,5. A los demás equipos que no queden en primera posición se aplicará la fórmula de *linspace* para calcular el valor obtenido.

8. Entregables

1. El notebook **Practica_Deteccion_Anomalias_CASAS.ipynb** ejecutado (o el código equivalente en un repositorio): vuestra codificación del tensor, vuestro detector, y la tabla de resultados de resultados.json, todo reproducible.
2. **pred_test.json** con las 40 probabilidades del test conjunto (sección 5.1): archivo del ranking de la clase (un **pred_test_*.json** por arquitectura si hacéis el +1 opcional).
3. Informe **PDF** con las secciones del Paso 5, incluyendo la tabla de métricas con el formato de la sección 6 (**máximo 5 páginas**).
4. Al menos una figura didáctica comparando una ejecución normal y una con error a nivel de sensor.
5. **Para el +1:** tabla de ranking de arquitecturas (la de ranking.json) con su justificación.

9. Rúbrica

Criterio	Ponderación
Preprocesado correcto y reproducible (tensor canales × tiempo, <i>carry-forward</i>)	20 %
Entrenamiento de al menos una arquitectura con protocolo y métricas bien definidos (AUC)	30 %
Validación honesta: split por participante, sin fuga, semilla fija	20 %
Figura a nivel de sensor (normal vs. error) y análisis de qué detecta / no detecta el modelo	20 %
Calidad del informe: claridad, limitaciones, citas	10 %
Extra: ranking de ≥ 2 arquitecturas con mismo protocolo + interpretación	+1 pt

10. Referencias

Cook, D. & Schmitter-Edgecombe, M. (2009). Assessing the quality of activities in a smart environment. *Methods of Information in Medicine*, 48(5), 480–485. <https://doi.org/10.3414/ME0592>

· Dataset: CASAS Smart Home dataset - scripted activities, with and without activity errors. Zenodo record 15712834. <https://zenodo.org/records/15712834>

· Más datasets CASAS: <https://zenodo.org/communities/casas>.